

以深度學習結合Zernike 參數辨識之平頂光束自適應校正研究

陳子彬

April 22, 2026

1 研究背景與問題描述

本研究的核心目標，是透過空間液晶調製器（Spatial Light Modulator, SLM）調製射入波的相位，使反射波在CCD上形成接近平頂（flat-top）的光強分布。理想情況下，平頂光束應具有均勻、平坦且邊界可控的強度輪廓，然而在實際實驗中，常受到失焦、散光、球差、像差耦合、光學元件裝配誤差以及環境擾動等非理想因素影響，使得最終量測到的光強分布存在起伏與不平整。

目前系統雖已能產生接近平頂的分布，但不平整度難以低於1%，顯示僅依靠人工調整相位參數已接近極限。尤其當補償模型使用Zernike polynomials 描述非理想波前時，若需要調整的參數數量達15 維，便會面臨高維搜尋空間過大、參數彼此耦合以及人工判讀效率過低的問題。因此，本研究希望發展一套結合深度學習與閉迴路控制的自適應調整方法，讓系統能夠由單張CCD 光強圖自動辨識其所對應的Zernike 畸變成分，進一步更新補償參數並反覆迭代，最終突破現有人工調整的效能上限。

2 研究動機

Zernike 多項式是描述圓孔徑波前誤差的標準工具，在光學工程中廣泛用於表示defocus、astigmatism、coma、spherical aberration 等常見像差。由於不同像差模式會在焦平面或成像面上留下特定的強度分布特徵，因此若能訓練一個模型，讓它直接從CCD 量測到的強度圖回推15 維Zernike 係數，即可將原本高度依賴經驗的人工調整，轉化為資料驅動的快速辨識與自動補償流程。

深度學習特別適合處理此類「影像輸入、連續參數輸出」的問題。相較於手工萃取特徵或逐一掃描參數，卷積神經網路（CNN）可以自動學習光斑紋理、局部對稱性、邊界變形與能量分布等特徵，並將它們映射到對應的Zernike 係數空間。因此，本研究不將問題視為分類，而是定義為**多輸出回歸**：輸入一張灰階CCD 強度圖，輸出一個15 維連續向量

$$\hat{\mathbf{z}} = (\hat{z}_1, \hat{z}_2, \dots, \hat{z}_{15}),$$

其中 $\hat{z}_i$ 表示第 i 個Zernike mode 的估計係數。藉此，模型不僅能做一次性的誤差診斷，也可作為自適應控制迴路中的估測器（estimator），在每次量測後提出修正方向。

3 研究目標

本研究擬達成以下四項目標：

1. 建立可描述平頂光束與Zernike 畸變之模擬資料產生器，系統性生成訓練所需的光強圖與參數標籤。
2. 設計並訓練一個以CNN 為核心的多輸出回歸模型，使其能由單張強度圖預測15 維Zernike 係數。
3. 建立自適應迭代校正流程，利用模型輸出更新SLM 補償參數，逐步降低平頂光束的不平整度。
4. 將模擬架構延伸至實驗平台，最終實現可上機運作的closed-loop 自適應光場校正系統。

4 研究方法

4.1 系統概念

整體系統可分為三個層次：

- (a) **前向光學模型 (forward model)**：描述給定Zernike 係數後，SLM 相位調製如何影響CCD 量測到的平頂強度分布。
- (b) **深度學習估測器**：由量測圖像回推隱含其中的Zernike 畸變成分。
- (c) **自適應控制器**：根據估測結果更新補償向量，使系統逐步逼近理想平頂分布。

此三者結合後，即形成從「觀測」到「辨識」再到「補償」的完整閉迴路架構。

4.2 訓練資料的模擬生成

研究第一步是建立資料集。由於實驗初期通常缺乏大量帶標籤的真實量測資料，因此將先以模擬方式生成訓練集。其基本流程如下：

1. 定義15 個欲補償的Zernike modes 與其係數範圍。
2. 在每次生成樣本時，從各係數範圍中隨機取樣，形成一組15 維參數向量。
3. 將各mode 線性組合成總波前誤差，疊加於理想平頂光束相位上。
4. 經由前向模型計算對應的CCD 強度圖，並加入合理的噪聲、位移、增益偏差與解析度限制，以模擬實際量測情況。

5. 以生成之強度圖作為輸入，以對應的15 維Zernike 係數作為標籤，建立監督式學習資料集。

值得注意的是，訓練資料不應只侷限於單一mode 單獨擾動，而應以15 維隨機疊加的方式生成。原因在於真實實驗中的像差通常同時存在，且彼此效應會交互耦合。若模型只看過單一mode 的樣本，將難以應對實際環境中的混合畸變。

4.3 CNN 回歸模型架構

本研究採用CNN 作為基礎模型，理由是光強圖中的局部紋理、邊緣變形與空間分布特徵，都適合透過卷積層進行抽取。模型的輸入與輸出形式如下：

- 輸入： $1 \times H \times W$ 的單通道灰階CCD 強度圖。
- 輸出：15 維連續值向量，對應15 個Zernike 係數。

一個可直接執行的demo 架構如下：

1. 4 層卷積模組，逐層抽取低階到高階空間特徵。
2. 每層搭配非線性活化函數與池化操作，以提升模型對局部特徵與尺度變化的描述能力。
3. 經過flatten 後接上數層全連接層，將影像特徵映射至15 維係數空間。
4. 輸出層不使用分類機率，而直接輸出15 個連續值。

在損失函數方面，選擇SmoothL1Loss 作為初始設定。相較於單純的MSE，SmoothL1 對少數大誤差樣本較為穩定，可降低訓練初期因個別難例造成的不穩定振盪。未來若要進一步強化模型對最終平頂品質的敏感度，也可設計混合損失函數，例如：

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{coeff}} + \lambda_2 \mathcal{L}_{\text{flatness}},$$

其中 $\mathcal{L}_{\text{coeff}}$ 為係數預測誤差， $\mathcal{L}_{\text{flatness}}$ 則根據重建或補償後的光強均勻度定義，使模型不僅學會回推係數，也朝向降低平頂不平整度的最終應用目標。

4.4 訓練流程

模型訓練將依下列步驟進行：

1. 利用模擬器大量生成訓練集、驗證集與測試集。
2. 將光強圖正規化後輸入CNN，輸出預測的15 維係數。
3. 以SmoothL1Loss 計算預測係數與真值之差異。

4. 使用Adam 或AdamW 作為最佳化器，進行多epoch 訓練。
5. 透過驗證集監控loss、每一維係數誤差以及整體預測穩定性。
6. 於測試階段檢視模型對未見樣本的泛化能力，並分析哪些mode 較容易或較困難辨識。

在報告與demo 中，可同步輸出訓練曲線、單張測試圖之預測結果、真值與預測值比較，以及補償前後的強度分布圖，作為模型有效性的佐證。

4.5 自適應迭代校正機制

本研究的重點不僅在於「辨識」，更在於「辨識後能立即調整」。因此在模型完成訓練後，將其嵌入自適應迭代控制流程。每次CCD 取得新的強度圖後，模型估測當前殘留畸變為 $\hat{\mathbf{z}}^{(k)}$ ，再依以下更新式修正補償向量：

$$\mathbf{z}_{\text{corr}}^{(k+1)} = \mathbf{z}_{\text{corr}}^{(k)} - \eta \hat{\mathbf{z}}^{(k)},$$

其中 η 為更新步長。若估測結果正確，則系統每次迭代都會削弱當前像差成分，使平頂分布逐步趨於均勻。此方法的優點在於：

- 不需人工逐一掃描15 維參數；
- 可快速回應環境擾動或系統漂移；
- 容易與現有SLM 控制流程整合，形成真正可上機的closed-loop 校正。

5 Zernike CNN Demo 程式設計與實作說明

為了讓研究構想不只停留在方法描述層次，本研究同步規劃一份可直接執行的PyTorch 範例程式zernike_cnn_demo.py，作為整個研究流程的prototype。此demo 的目的，不是立即完整重現實驗室中的SLM 與傅立葉光學傳播系統，而是先建立一條可驗證的深度學習工作流，將「資料生成」、「模型訓練」、「係數預測」與「自適應校正」四個模組串接起來，使研究者能快速檢驗由單張CCD 光強圖反推Zernike 係數的可行性。

此程式的核心功能可概括為五項：

1. 建立理想平頂光強分布作為target intensity map。
2. 使用15 個Zernike 係數，透過surrogate forward model 產生受像差影響的失真光強圖。
3. 訓練一個CNN 回歸模型，將輸入影像映射為15 維係數向量。
4. 示範adaptive correction 流程，重複估測殘差像差並逐步扣除。
5. 輸出訓練曲線、預測結果、校正前後比較圖與訓練後模型權重。

5.1 Demo 的定位與研究意義

此demo 屬於「可執行研究原型」，其價值在於先驗證機器學習pipeline 是否成立，而非直接取代真實光學系統。程式中所使用的FlatTopZernikeSimulator.simulate() 是一個surrogate model，它用空間扭曲、模糊、陰影變化與雜訊等方式模擬Zernike 像差對平頂光斑的影響，因此具有物理直觀性，但仍不是實驗平台上精確的SLM+CCD forward model。

換言之，這份demo 可用來回答下列關鍵研究問題：

- 若輸入為單張強度圖，CNN 是否能有效學會對應的15 維Zernike 係數？
- 在surrogate model 下訓練出的估測器，是否能支援閉迴路adaptive correction？
- 若整體流程成立，後續要替換成真實光學模型時，哪些模組可以直接沿用？

因而，此demo 在研究流程中的角色可視為正式上機前的驗證平台。完成此平台後，後續只需將模擬器替換為真實的光學傳播計算，或直接改用已知Zernike 設定下量測到的CCD 影像，即可逐步銜接到實驗應用。

5.2 程式整體架構

zernike_cnn_demo.py 的程式架構可分為七個模組：

1. 參數與模式定義：設定亂數種子、計算裝置、15 個Zernike modes 名稱與對應階數。
2. Zernike 基底生成：建立圓孔徑座標網格，計算Zernike radial polynomial 與完整基底函數。
3. 前向模擬器：由理想平頂target 出發，根據係數生成失真影像。
4. 資料集模組：隨機生成大量係數向量與對應影像，組成監督式訓練資料。
5. CNN 回歸模型：以卷積層與全連接層構成多輸出回歸網路。
6. 訓練與推論模組：包含epoch 訓練函式、驗證流程與單張影像係數預測。
7. 自適應校正與視覺化：執行iterative correction，並輸出訓練曲線與比較圖。

如此設計的優點是模組邊界清楚。未來若要將demo 升級為實驗版本，最主要需更換的是前向模擬器與資料來源，其餘如資料流、模型架構、訓練與adaptive loop 可大致保留。

5.3 15 個Zernike 模式的定義

程式中選用15 個non-piston Zernike modes，涵蓋傾斜、散光、離焦、彗差、三葉像差、球差與更高階模式。其目的在於建立一個足以描述多種實驗常見非理想效應的參數空間。模式定義如下：

索引	模式名稱
0	tilt_y
1	tilt_x
2	astig_45
3	defocus
4	astig_0
5	trefoil_y
6	coma_y
7	coma_x
8	trefoil_x
9	quadrafoil_y
10	secondary_astig_45
11	primary_spherical
12	secondary_astig_0
13	quadrafoil_x
14	secondary_coma_y

在程式中，每個Zernike mode 都會先被標準化，使其在圓孔徑內具有可比較的尺度。這樣做的好處，是讓不同模式的係數大小更具有一致性，也讓神經網路的學習更穩定。

5.4 Surrogate forward model 的設計邏輯

FlatTopZernikeSimulator 是此demo 的核心。它的流程不是直接計算完整繞射傳播，而是以「具光學意義的近似失真機制」來模擬實驗中可能觀察到的強度變形。其步驟如下：

1. 先建立理想平頂光斑作為target image。
2. 將輸入的15 維Zernike 係數與基底做線性組合，形成總波前誤差 $W(x, y)$ 。
3. 對波前取空間梯度，得到影像空間中的位移場 dx, dy ，模擬像差引起的幾何扭曲。
4. 依據係數RMS 大小決定高斯模糊強度，對應成像失焦或高階畸變造成的模糊。
5. 使用波前的Laplacian 建立shading 項，反映局部能量分布與曲率造成的亮度調變。
6. 最後加入隨機雜訊，使資料更接近真實CCD 量測。

這種surrogate 設計雖然不是完整物理模型，但它可以保留「不同Zernike 係數造成不同空間紋理變化」這項重要特性，因此足以支撐CNN 對像差成分的辨識學習。

5.5 資料集生成方式

ZernikeDataset 類別會批次隨機產生係數向量，再透過simulator 生成對應影像。每一筆樣本都包含：

- 輸入：大小為 $1 \times 64 \times 64$ 的單通道光強圖。
- 標籤：15 維連續Zernike 係數。

demo 預設生成800 筆資料，並切分為80% 訓練集與20% 驗證集。這樣的規模足以用來演示整體流程，也能在合理時間內完成訓練。未來若替換為更擬真的forward model，則可進一步擴充資料量與參數分布範圍，以提升模型泛化能力。

5.6 CNN 回歸模型細節

ZernikeRegressor 採用四層卷積特徵抽取架構，並在最後接上全連接回歸頭。其設計理念如下：

1. 第一至第四層卷積模組逐步將通道數從1 提升至128，以擷取從局部亮度變化到高階空間結構的特徵。
2. 每個ConvBlock 包含兩層卷積、Batch Normalization 與ReLU，可提升特徵表達與訓練穩定性。
3. 中間穿插MaxPool2d 進行降採樣，使模型學會較大尺度的空間模式。
4. 最後使用AdaptiveAvgPool2d((1,1)) 壓縮空間維度，接著以全連接層輸出15 個連續值。
5. 在回歸head 中加入少量Dropout，可降低過擬合風險。

此模型屬於中小型CNN，優點是結構清楚、參數量適中、容易訓練，也方便未來依需求升級為更深的backbone，例如ResNet 類型結構。

5.7 訓練流程與損失函數

程式中的訓練函式train_model() 採用以下設定：

- 損失函數：SmoothL1Loss
- 最佳化器：Adam
- 學習率排程：CosineAnnealingLR
- 訓練epoch：demo 預設為5
- 批次大小：64

在每一個epoch 中，程式都會同時計算訓練集與驗證集上的loss 與mean absolute error。這種設計有兩個目的：第一，可即時觀察模型是否收斂；第二，可透過驗證表現保存最佳權重，避免單純使用最後一輪參數而造成性能波動。

由於此問題本質上是連續參數估測，因此不適合用分類交叉熵處理，而應使用回歸型損失。SmoothL1Loss 相較MSE 更不易受少數極端誤差支配，適合用於此類多維係數回歸任務。

5.8 單張影像係數預測

完成訓練後，可透過`predict_coeffs()` 將單張影像轉為模型輸入，並輸出15 維預測向量。這對本研究非常重要，因為實驗上每次量測CCD 後，控制器所需的的就是這組估測係數。換句話說，模型在整個自適應系統中的角色，相當於即時波前像差判讀器。

5.9 自適應校正demo

程式中的`run_adaptive_loop()` 進一步展示模型如何被嵌入控制迴路。其更新機制可寫為：

$$\mathbf{c}^{(k+1)} = \mathbf{c}^{(k)} - g \hat{\mathbf{z}}^{(k)},$$

其中 $\mathbf{c}^{(k)}$ 為第 k 次迭代時的補償向量， g 為增益， $\hat{\mathbf{z}}^{(k)}$ 為模型對當前殘留像差的預測。demo 會在每次迭代中紀錄三項資訊：

1. 目前的殘差係數RMS。
2. 平頂區域的不平整度指標（flatness non-uniformity）。
3. 每一步所生成的光強圖像。

若模型估測方向正確，則殘差係數RMS 與平頂不平整度都應隨迭代逐步下降。這正是本研究希望在真實實驗中達成的closed-loop 改善效果。

5.10 輸出結果與可視化

此demo 會自動輸出多種圖表與模型檔案，包含：

- `training_curve.png`：呈現訓練與驗證loss 的收斂情形。
- `prediction_example.png`：展示理想target、輸入失真圖，以及各mode 的預測誤差。
- `adaptive_correction.png`：比較校正前後影像，並呈現flatness 與殘差的迭代趨勢。
- `zernike_cnn_model.pt`：保存訓練後模型權重、模式名稱與模擬設定。

原始demo 指定輸出至`/mnt/data/demo_outputs/`。若在實驗室或本地研究環境中使用，亦可依需求改成研究資料夾中的相對路徑，以利與量測資料、訓練紀錄及版本控制整合。

5.11 執行方式與環境需求

此程式的執行方式非常直接，只需在安裝好numpy、scipy、matplotlib與torch的Python環境中執行：

```
python zernike_cnn_demo.py
```

程式會自動判斷是否有可用GPU，若無則使用CPU。對於demo規模的資料與模型而言，即使在CPU上也可完成示範性訓練，只是運行時間會較長。

5.12 如何銜接到真實實驗系統

從研究落地的角度來看，這份demo最重要的不是單次執行結果，而是它指出了未來擴充的方向。若要將其轉換為真正可上機的研究工具，主要需要進行下列三項修改：

1. 將FlatTopZernikeSimulator.simulate() 替換為真實的光學傳播模型，例如SLM 相位調製加上傅立葉光學propagation的數值模擬。
2. 將訓練資料改為更貼近實驗的資料來源，包括真實CCD ROI、量測噪聲、曝光變化以及已知Zernike設定下取得的校正樣本。
3. 將訓練目標由單純係數誤差，進一步擴充成「係數估測+ 平頂品質」的混合目標，使模型更直接朝最終實驗指標最佳化。

因此，這份demo不只是教學程式，更是一個研究開發的起點。它將抽象的深度學習想法具體化為可執行、可觀測、可逐步替換的工程流程，能有效降低從概念到實作之間的落差。

6 預期成果

本研究預期完成以下成果：

1. 建立一套可直接執行的PyTorch CNN demo，內容包含Zernike參數定義、資料模擬、模型訓練、係數預測與自適應校正流程。
2. 驗證「單張光強圖回推15維Zernike係數」的可行性，並分析不同mode對辨識精度的影響。
3. 建立校正前後之平頂光強比較機制，以不平整度、均勻度與收斂速度作為量化指標。
4. 為後續導入真實SLM相位函式、CCD ROI與實驗量測資料微調（fine-tuning）奠定基礎。

若模型與控制迴路運作順利，預期可較人工調整更穩定且更快速地壓低平頂不平整度，並朝突破1%門檻前進。

7 研究創新性與貢獻

本研究的創新性主要體現在以下三點：

1. 將波前像差校正問題轉化為影像到參數的回歸問題：以深度學習取代傳統依賴人工經驗的參數調整流程。
2. 將Zernike 模型與CNN 辨識器結合：使物理可解釋的像差表示法與資料驅動模型互補，兼顧可解釋性與實用性。
3. 導入閉迴路自適應校正：不只離線分析影像，而是讓模型直接參與實驗控制與即時補償。

此研究若能完成，除了有助於提升平頂光束品質，也可作為其他光學整形、波前校正與智慧光學控制問題的技術原型。

8 可能遭遇的挑戰與因應策略

雖然此方法具有高度潛力，但在實作上仍可能遭遇若干挑戰：

1. 模擬與真實實驗存在domain gap：若模擬器與實際光學系統差異過大，模型可能在實驗資料上性能下降。對策是逐步將真實光學機制、噪聲、ROI 裁切與儀器非理想性納入forward model，並於後期使用真實量測資料進行微調。
2. 不同Zernike mode 之間存在耦合：部分像差可能產生相似的光強特徵，造成辨識混淆。對策是提高資料多樣性、強化混合樣本比例，並評估是否需要更深的backbone 或加入殘差結構。
3. 控制迴路可能收斂不穩：若更新步長過大，可能導致過度補償與震盪。對策是設計步長衰減、信心權重或混合PID/學習式控制策略，以提升實驗穩定性。

9 研究進度規劃

本研究可初步規劃為三個階段：

階段	工作內容
第一階段	建立15 維Zernike 模擬器、定義平頂光束forward model、產生訓練資料並完成baseline CNN。
第二階段	完成模型訓練與驗證，分析各mode 的辨識表現，建立單張圖回推係數與校正前後視覺化比較工具。
第三階段	將模型整合至實驗平台，串接實際SLM phase map 與CCD 量測，進行closed-loop 自適應補償測試。

10 結論

綜合而言，本研究提出一套以深度學習為核心的平頂光束自適應校正方案，目標在於解決傳統人工調整15 維Zernike 參數時效率低、難以尋找最佳解的瓶頸。透過模擬資料建立、CNN 多輸出回歸模型訓練，以及closed-loop 參數更新機制，本研究希望讓系統具備由單張CCD 強度圖自動判讀波前誤差並即時修正的能力。

此計畫的價值不僅在於提供一個可直接執行的demo，更在於建立一條明確可擴充的技術路徑：先驗證深度學習能否從光強圖回推Zernike 係數，再逐步替換成更接近實驗的forward model，最後以真實量測資料進行微調與上機驗證。若研究成果順利落地，將有望突破目前平頂光束不平整度難以低於1% 的限制，並為智慧化光學控制建立具代表性的研究範例。

A Zernike CNN Demo 完整程式碼

本附錄收錄本研究使用之zernike_cnn_demo.py 完整程式碼。為了提升可讀性，程式中已補上補充註解，說明各模組在資料生成、模型訓練與自適應校正流程中的功能。若後續需銜接真實實驗系統，可優先修改程式中的FlatTopZernikeSimulator.simulate() 與資料來源模組。

```
1  import os
2  os.environ.setdefault('MPLCONFIGDIR', '/tmp/mplconfig')
3  import math
4  import random
5  from dataclasses import dataclass
6  from typing import List, Tuple
7
8  import matplotlib
9  matplotlib.use('Agg')
10 import matplotlib.pyplot as plt
11 import numpy as np
12 from scipy.ndimage import gaussian_filter, laplace, map_coordinates
13
14 import torch
15 import torch.nn as nn
16 from torch.utils.data import DataLoader, Dataset, random_split
17
18 # Fix random seeds for reproducibility.
19 SEED = 42
20 random.seed(SEED)
21 np.random.seed(SEED)
22 torch.manual_seed(SEED)
23 DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
24
25 # Fifteen non-piston Zernike modes.
26 # Each tuple is (n, m, name) for one aberration mode.
27 ZERNIKE_MODES: List[Tuple[int, int, str]] = [
28     (1, -1, 'tilt_y'),
29     (1, 1, 'tilt_x'),
30     (2, -2, 'astig_45'),
31     (2, 0, 'defocus'),
32     (2, 2, 'astig_0'),
33     (3, -3, 'trefoil_y'),
34     (3, -1, 'coma_y'),
35     (3, 1, 'coma_x'),
```

```

36     (3, 3, 'trefoil_x'),
37     (4, -4, 'quadrafoil_y'),
38     (4, -2, 'secondary_astig_45'),
39     (4, 0, 'primary_spherical'),
40     (4, 2, 'secondary_astig_0'),
41     (4, 4, 'quadrafoil_x'),
42     (5, -1, 'secondary_coma_y'),
43 ]
44
45
46 def make_grid(n: int = 64):
47     # Build a normalized coordinate grid over [-1, 1] x [-1, 1].
48     x = np.linspace(-1.0, 1.0, n)
49     y = np.linspace(-1.0, 1.0, n)
50     xx, yy = np.meshgrid(x, y)
51     rr = np.sqrt(xx**2 + yy**2)
52     th = np.arctan2(yy, xx)
53     pupil = rr <= 1.0
54     return xx, yy, rr, th, pupil
55
56
57 def zernike_radial(n: int, m: int, r: np.ndarray) -> np.ndarray:
58     # Compute the radial part of a Zernike polynomial.
59     m = abs(m)
60     if (n - m) % 2 != 0:
61         return np.zeros_like(r)
62     out = np.zeros_like(r)
63     for k in range((n - m) // 2 + 1):
64         coeff = ((-1) ** k) * math.factorial(n - k)
65         denom = (
66             math.factorial(k)
67             * math.factorial((n + m) // 2 - k)
68             * math.factorial((n - m) // 2 - k)
69         )
70         out += coeff / denom * r ** (n - 2 * k)
71     return out
72
73
74 def zernike(n: int, m: int, r: np.ndarray, theta: np.ndarray) -> np.ndarray:
75     # Combine radial and angular parts and normalize inside the pupil.
76     rad = zernike_radial(n, m, r)
77     if m > 0:
78         z = rad * np.cos(m * theta)
79     elif m < 0:
80         z = rad * np.sin(abs(m) * theta)
81     else:
82         z = rad
83     mask = r <= 1.0
84     z = z * mask
85     std = z[mask].std()
86     if std > 1e-8:
87         z = z / std
88     return z.astype(np.float32)
89
90
91 @dataclass
92 class ForwardModelConfig:
93     # Image size and surrogate forward-model distortion parameters.
94     image_size: int = 64

```

```

95     coeff_std: float = 0.18
96     warp_scale: float = 0.055
97     blur_scale: float = 1.20
98     shading_scale: float = 0.10
99     noise_std: float = 0.005
100    target_order: int = 10
101    target_wx: float = 0.43
102    target_wy: float = 0.25
103
104
105    class FlatTopZernikeSimulator:
106        """
107        Demo surrogate model.
108        It starts from an ideal flat-top target image and uses Zernike coefficients
109        to induce physically motivated warp/blur/shading distortions.
110
111        In your real system, replace simulate() with either:
112        1) your full Fourier-optics + SLM forward model, or
113        2) real CCD measurements collected under known Zernike settings.
114        """
115        def __init__(self, cfg: ForwardModelConfig):
116            self.cfg = cfg
117            self.xx, self.yy, self.rr, self.th, self.pupil = make_grid(cfg.image_size)
118            self.target = self._make_target()
119            self.basis = self._build_basis()
120            # Precompute spatial gradients for later warp-field construction.
121            self.basis_dx = np.stack([np.gradient(b, axis=1) for b in self.basis], axis=0)
122            self.basis_dy = np.stack([np.gradient(b, axis=0) for b in self.basis], axis=0)
123
124        def _make_target(self):
125            # Build an ideal flat-top target using a high-order super-Gaussian.
126            wx, wy, p = self.cfg.target_wx, self.cfg.target_wy, self.cfg.target_order
127            tgt = np.exp(-((np.abs(self.xx) / wx) ** p + (np.abs(self.yy) / wy) ** p))
128            return (tgt / tgt.max()).astype(np.float32)
129
130        def _build_basis(self):
131            return np.stack([zernike(n, m, self.rr, self.th) for n, m, _ in ZERNIKE_MODES], axis=0)
132
133        def simulate(self, coeffs: np.ndarray, add_noise: bool = True):
134            # Project the 15D coefficient vector onto the Zernike basis.
135            coeffs = np.asarray(coeffs, dtype=np.float32)
136            wavefront = np.tensordot(coeffs, self.basis, axes=(0, 0))
137            dx = np.tensordot(coeffs, self.basis_dx, axes=(0, 0))
138            dy = np.tensordot(coeffs, self.basis_dy, axes=(0, 0))
139
140            # Use wavefront gradients to create spatial warping in the image.
141            xw = self.xx + self.cfg.warp_scale * dx
142            yw = self.yy + self.cfg.warp_scale * dy
143            px = (xw + 1.0) * 0.5 * (self.cfg.image_size - 1)
144            py = (yw + 1.0) * 0.5 * (self.cfg.image_size - 1)
145            warped = map_coordinates(self.target, [py, px], order=1, mode='constant', cval=0.0)
146
147            # Set blur strength from coefficient RMS to mimic defocus/high-order blur.
148            blur_sigma = self.cfg.blur_scale * float(np.sqrt(np.mean(coeffs**2)))
149            if blur_sigma > 1e-6:
150                warped = gaussian_filter(warped, sigma=blur_sigma)
151
152            # Use the Laplacian to create a local shading modulation term.
153            shading = 1.0 + self.cfg.shading_scale * laplace(wavefront)

```

```

154     shading = np.clip(shading, 0.75, 1.25)
155     image = np.clip(warped * shading, 0.0, None)
156     image = image / (image.max() + 1e-8)
157
158     # Add sensor noise so the synthetic image better resembles CCD data.
159     if add_noise:
160         image = image + np.random.normal(0.0, self.cfg.noise_std, size=image.shape).astype(
161             np.float32)
162         image = np.clip(image, 0.0, 1.0)
163     return image.astype(np.float32)
164
165     def flatness_metric(self, image: np.ndarray, roi_level: float = 0.8) -> float:
166         # Compute relative standard deviation over the flat-top ROI as a flatness metric.
167         mask = self.target > roi_level
168         region = image[mask]
169         return float(region.std() / (region.mean() + 1e-8))
170
171     class ZernikeDataset(Dataset):
172         def __init__(self, simulator: FlatTopZernikeSimulator, num_samples: int = 1000):
173             self.sim = simulator
174             self.num_modes = len(ZERNIKE_MODES)
175             # Randomly sample coefficient vectors and synthesize one image per sample.
176             self.coefts = np.random.normal(0.0, self.sim.cfg.coeft_std, size=(num_samples, self.
177                 num_modes)).astype(np.float32)
178             self.images = np.stack([self.sim.simulate(c, add_noise=True) for c in self.coefts], axis
179                 =0)[: , None, :, :]
180
181         def __len__(self):
182             return len(self.coefts)
183
184         def __getitem__(self, idx):
185             return torch.from_numpy(self.images[idx]), torch.from_numpy(self.coefts[idx])
186
187     class ConvBlock(nn.Module):
188         def __init__(self, c_in, c_out):
189             super().__init__()
190             self.net = nn.Sequential(
191                 nn.Conv2d(c_in, c_out, 3, padding=1),
192                 nn.BatchNorm2d(c_out),
193                 nn.ReLU(inplace=True),
194                 nn.Conv2d(c_out, c_out, 3, padding=1),
195                 nn.BatchNorm2d(c_out),
196                 nn.ReLU(inplace=True),
197             )
198
199         def forward(self, x):
200             return self.net(x)
201
202     class ZernikeRegressor(nn.Module):
203         def __init__(self, num_modes: int):
204             super().__init__()
205             # Four-layer convolutional backbone followed by a 15-value regression head.
206             self.features = nn.Sequential(
207                 ConvBlock(1, 16),
208                 nn.MaxPool2d(2),
209                 ConvBlock(16, 32),

```

```

210         nn.MaxPool2d(2),
211         ConvBlock(32, 64),
212         nn.MaxPool2d(2),
213         ConvBlock(64, 128),
214         nn.AdaptiveAvgPool2d((1, 1)),
215     )
216     self.head = nn.Sequential(
217         nn.Flatten(),
218         nn.Linear(128, 128),
219         nn.ReLU(inplace=True),
220         nn.Dropout(0.15),
221         nn.Linear(128, num_modes),
222     )
223
224     def forward(self, x):
225         return self.head(self.features(x))
226
227
228     def run_epoch(model, loader, criterion, optimizer=None):
229         training = optimizer is not None
230         model.train(training)
231         total_loss = 0.0
232         total_mae = 0.0
233         total_n = 0
234
235         for x, y in loader:
236             x = x.to(DEVICE)
237             y = y.to(DEVICE)
238             pred = model(x)
239             loss = criterion(pred, y)
240             if training:
241                 optimizer.zero_grad()
242                 loss.backward()
243                 optimizer.step()
244             mae = (pred - y).abs().mean().item()
245             bs = x.size(0)
246             total_loss += loss.item() * bs
247             total_mae += mae * bs
248             total_n += bs
249         return total_loss / total_n, total_mae / total_n
250
251
252     def train_model(model, train_loader, val_loader, epochs=5, lr=1e-3):
253         criterion = nn.SmoothL1Loss()
254         optimizer = torch.optim.Adam(model.parameters(), lr=lr)
255         scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
256
257         history = {'train_loss': [], 'val_loss': [], 'train_mae': [], 'val_mae': []}
258         best_state = None
259         best_val = float('inf')
260
261         for ep in range(1, epochs + 1):
262             train_loss, train_mae = run_epoch(model, train_loader, criterion, optimizer)
263             val_loss, val_mae = run_epoch(model, val_loader, criterion, optimizer=None)
264             scheduler.step()
265             history['train_loss'].append(train_loss)
266             history['val_loss'].append(val_loss)
267             history['train_mae'].append(train_mae)
268             history['val_mae'].append(val_mae)

```

```

269         print(f'Epoch {ep:02d}/{epochs} | train loss={train_loss:.5f}, val loss={val_loss:.5f},
270               train mae={train_mae:.5f}, val mae={val_mae:.5f}')
271         if val_loss < best_val:
272             best_val = val_loss
273             best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
274         if best_state is not None:
275             model.load_state_dict(best_state)
276         return history
277
278     def predict_coeffs(model: nn.Module, image: np.ndarray):
279         # Run inference on one image and return the 15 predicted coefficients.
280         model.eval()
281         with torch.no_grad():
282             x = torch.from_numpy(image[None, None, :, :]).float().to(DEVICE)
283             pred = model(x).cpu().numpy()[0]
284         return pred.astype(np.float32)
285
286     def run_adaptive_loop(sim: FlatTopZernikeSimulator, model: nn.Module, true_coeffs: np.ndarray,
287                          steps: int = 8, gain: float = 0.85):
288         # Closed-loop correction: repeatedly estimate residual aberration and update compensation.
289         correction = np.zeros_like(true_coeffs, dtype=np.float32)
290         images = []
291         residual_norms = []
292         flatness = []
293         predicted = []
294
295         for _ in range(steps):
296             residual = true_coeffs + correction
297             image = sim.simulate(residual, add_noise=False)
298             pred = predict_coeffs(model, image)
299             correction = correction - gain * pred
300             images.append(image)
301             residual_norms.append(float(np.sqrt(np.mean(residual**2))))
302             flatness.append(sim.flatness_metric(image))
303             predicted.append(pred)
304
305         final_residual = true_coeffs + correction
306         final_image = sim.simulate(final_residual, add_noise=False)
307         images.append(final_image)
308         residual_norms.append(float(np.sqrt(np.mean(final_residual**2))))
309         flatness.append(sim.flatness_metric(final_image))
310
311         return {
312             'images': images,
313             'residual_norms': residual_norms,
314             'flatness': flatness,
315             'predicted': predicted,
316             'final_residual': final_residual,
317             'final_correction': correction,
318         }
319
320     def plot_training(history, outdir):
321         plt.figure(figsize=(7, 4))
322         plt.plot(history['train_loss'], label='train loss')
323         plt.plot(history['val_loss'], label='val loss')
324         plt.xlabel('epoch')

```

```

326 plt.ylabel('SmoothL1 loss')
327 plt.legend()
328 plt.tight_layout()
329 plt.savefig(os.path.join(outdir, 'training_curve.png'), dpi=180)
330 plt.close()
331
332
333 def plot_prediction_example(sim, model, dataset, outdir):
334     idx = np.random.randint(len(dataset))
335     image = dataset.images[idx, 0]
336     true_coeffs = dataset.coeffs[idx]
337     pred_coeffs = predict_coeffs(model, image)
338
339     fig, axes = plt.subplots(1, 3, figsize=(12, 4))
340     axes[0].imshow(sim.target, cmap='inferno')
341     axes[0].set_title('ideal flat-top target')
342     axes[1].imshow(image, cmap='inferno')
343     axes[1].set_title('input distorted image')
344     axes[2].bar(np.arange(len(true_coeffs)), np.abs(true_coeffs - pred_coeffs))
345     axes[2].set_title('|prediction error| by mode')
346     axes[2].set_xlabel('mode index')
347     for ax in axes[:2]:
348         ax.axis('off')
349     plt.tight_layout()
350     plt.savefig(os.path.join(outdir, 'prediction_example.png'), dpi=180)
351     plt.close()
352
353
354 def plot_adaptive_result(sim, result, outdir):
355     initial = result['images'][0]
356     final = result['images'][-1]
357     steps = np.arange(len(result['residual_norms']))
358
359     fig, axes = plt.subplots(2, 3, figsize=(12, 8))
360     axes[0, 0].imshow(sim.target, cmap='inferno')
361     axes[0, 0].set_title('ideal target')
362     axes[0, 1].imshow(initial, cmap='inferno')
363     axes[0, 1].set_title(f'before correction\nflatness={result["flatness"][0]*100:.2f}%')
364     axes[0, 2].imshow(final, cmap='inferno')
365     axes[0, 2].set_title(f'after correction\nflatness={result["flatness"][-1]*100:.2f}%')
366     axes[1, 0].plot(steps, result['residual_norms'], marker='o')
367     axes[1, 0].set_title('residual coefficient RMS')
368     axes[1, 0].set_xlabel('iteration')
369     axes[1, 1].plot(steps, np.array(result['flatness']) * 100.0, marker='o')
370     axes[1, 1].set_title('flatness non-uniformity (%)')
371     axes[1, 1].set_xlabel('iteration')
372     axes[1, 2].bar(np.arange(len(result['final_residual'])), result['final_residual'])
373     axes[1, 2].set_title('final residual coefficients')
374     axes[1, 2].set_xlabel('mode index')
375     for ax in axes[0]:
376         ax.axis('off')
377     plt.tight_layout()
378     plt.savefig(os.path.join(outdir, 'adaptive_correction.png'), dpi=180)
379     plt.close()
380
381
382 def main():
383     # Use a relative output directory so results stay inside the workspace.
384     outdir = 'demo_outputs'

```

```

385     os.makedirs(outdir, exist_ok=True)
386
387     cfg = ForwardModelConfig()
388     sim = FlatTopZernikeSimulator(cfg)
389     dataset = ZernikeDataset(sim, num_samples=800)
390
391     n_train = int(0.8 * len(dataset))
392     n_val = len(dataset) - n_train
393     train_set, val_set = random_split(dataset, [n_train, n_val], generator=torch.Generator().
        manual_seed(SEED))
394     train_loader = DataLoader(train_set, batch_size=64, shuffle=True, num_workers=0)
395     val_loader = DataLoader(val_set, batch_size=64, shuffle=False, num_workers=0)
396
397     model = ZernikeRegressor(num_modes=len(ZERNIKE_MODES)).to(DEVICE)
398     print(f'Training on device: {DEVICE}')
399     history = train_model(model, train_loader, val_loader, epochs=5, lr=1e-3)
400
401     torch.save({
402         'model_state': model.state_dict(),
403         'mode_names': [name for _, _, name in ZERNIKE_MODES],
404         'config': cfg.__dict__,
405     }, os.path.join(outdir, 'zernike_cnn_model.pt'))
406
407     plot_training(history, outdir)
408     plot_prediction_example(sim, model, dataset, outdir)
409
410     true_coeffs = np.random.normal(0.0, cfg.coeff_std, size=len(ZERNIKE_MODES)).astype(np.
        float32)
411     result = run_adaptive_loop(sim, model, true_coeffs, steps=8, gain=0.85)
412     plot_adaptive_result(sim, result, outdir)
413
414     print('\nMode list:')
415     for i, (_, _, name) in enumerate(ZERNIKE_MODES):
416         print(f'{i:2d}: {name}')
417     print('\nTrue coefficients:')
418     print(np.round(true_coeffs, 4))
419     print('\nFinal residual coefficients:')
420     print(np.round(result['final_residual'], 4))
421     print(f"\nInitial flatness non-uniformity: {result['flatness'][0]*100:.2f}%")
422     print(f"Final flatness non-uniformity: {result['flatness'][-1]*100:.2f}%")
423     print(f'\nOutputs saved to: {outdir}')
424
425
426 if __name__ == '__main__':
427     main()

```